第8小时

数据库设计基础知识

8.0　章节考点分析

第 8 小时主要学习数据库基础概念、关系数据库、数据库设计、应用程序与数据库交互、NoSQL 数据库等内容。本小时内容侧重于概念知识，知识点会涉及单选题（约占 2～5 分）和案例题（25 分），根据以往全国计算机技术与软件专业技术资格（水平）考试的出题规律，考查的知识点多源于教材，扩展内容较少。本小时知识架构如图 8.1 所示。

图 8.1　本小时知识架构

【导读小贴士】

作为一名合格的系统架构师，必须掌握数据库设计的基本概念和方法。本小时介绍数据库技术的发展历程以及数据模型、主流的关系数据库、数据库设计的步骤与方法、新型数据库 NoSQL。系统架构设计师考试中的一些案例分析题会来自本小时的内容。建议考生在理解的基础上掌握核心知识点，学会灵活应用。

8.1 数据库基础概念

【基础知识点】

1. 基础概念

（1）数据（Data）：是描述事物的符号记录，它具有多种表现形式，如文字、图形、图像、声音和语言等。

（2）数据库系统（DataBase System，DBS）：是一个采用了数据库技术，有组织地、动态地存储大量相关联数据，从而方便多用户访问的计算机系统。

（3）数据库（DataBase，DB）：是统一管理的、长期储存在计算机内的，有组织的相关数据的集合。

（4）数据库管理系统（DataBase Management System，DBMS）：是数据库系统的核心软件，是由一组相互关联的数据集合和一组用以访问这些数据的软件组成。DBMS 通常分三类：关系数据库系统（Relation DataBase System，RDBS）、面向对象的数据库系统（Object-Oriented DataBase Systems，OODBS）、对象关系数据库系统（Objective Relational DataBase System，ORDBS）。

2. 发展阶段

（1）人工管理阶段。

特点：数据量较少、数据不保存、没有软件系统对数据进行处理。

缺点：应用程序与数据之间依赖性太强、数据组与数据组之间存在数据冗余。

（2）文件系统阶段。

特点：数据可长期保留、数据不属于某个特定应用、文件组织形式多样化。

缺点：数据冗余、数据不一致性、数据孤立。

（3）数据库系统阶段。

特点：采用复杂的数据模型表示数据结构、有较高的数据独立性。

优点：对应用程序的高度独立性、数据的充分共享性、操作方便性。

3. 数据模型

数据模型三要素：数据结构、数据操作、数据的约束条件。其中，数据的约束条件包括：

（1）实体完整性。实体完整性是指实体的主属性不能取空值。

（2）参照完整性。在关系数据库中主要是指外键参照的完整性。若 A 关系中的某个或者某些属性参照 B 关系或其他几个关系中的属性，那么在关系 A 中该属性要么为空，要么必须出现在 B 关系或者其他的关系的对应属性中。

（3）用户定义完整性。用户定义完整性反映的是某一个具体应用所对应的数据必须满足一定的约束条件。例如，软考成绩不能小于 0，也不能大于 75。

4. 数据库管理系统

数据库管理系统的主要功能包括数据定义，数据库操作，数据库运行管理，数据组织、存储和管理，数据库的建立和维护。数据库管理系统的特点：数据结构化且统一管理、有较高的数据独立性、数据控制功能。数据控制功能包括：保证护具的安全性和完整性以及并发控制的能力。

5. 数据库三级模式两级映像

数据库一般采用三级模式，其体系结构如图 8.2 所示，系统开发人员需要通过视图层、逻辑层和物理层三个层次上的抽象来降低用户屏蔽系统的复杂性，简化用户与系统的交互。从数据库管理系统的角度，数据库也分为外模式、概念模式和内模式。具体概念见表 8.1。

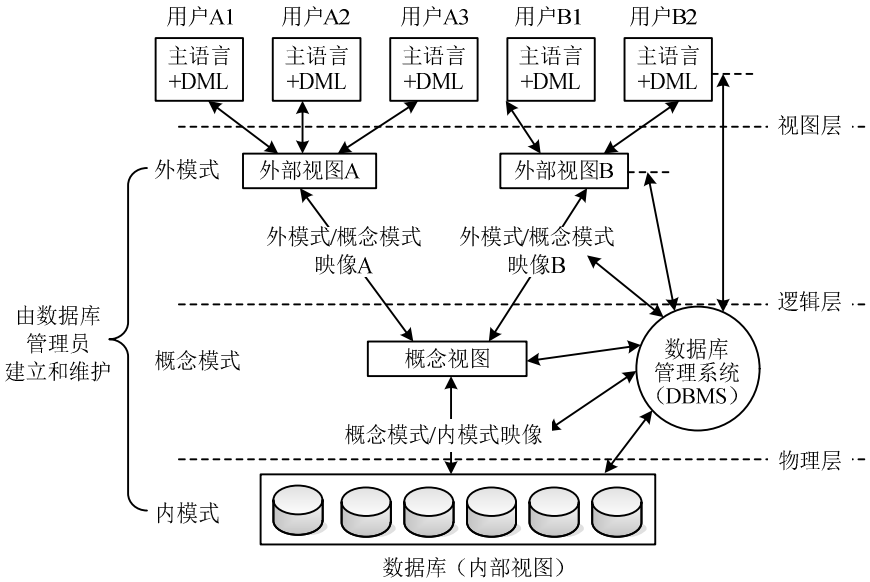

图 8.2　数据库系统体系结构

数据库系统在三级模式之间提供了两级映像：概念模式/内模式映像、外模式/概念模式映像。这两级映像保证了数据库中的数据具有较高的逻辑独立性和物理独立性。具体见表 8.2。

表 8.1 数据库的三级模式

概念模式	外模式	内模式
是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图	又叫子模式或用户模式，用以描述用户看到或使用的那部分数据的逻辑结构，用户根据外模式用数据操作语句或应用程序去操作数据库中的数据	是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式，定义所有的内部记录类型、索引和文件的组织方式等

表 8.2 数据库的两级映像

逻辑独立性	物理独立性
对应外模式和概念模式之间的映像。指应用程序与数据库中的逻辑结构独立，当数据的逻辑结构改变时，应用程序不变	对应概念模式和内模式之间的映像。指应用程序与磁盘中的数据互相独立。当数据的物理存储改变时，应用程序不变

8.2 关系数据库

【基础知识点】

1. 基本概念

（1）属性（Attribute）：在现实世界中，要描述一个事物常常取若干特征来表示。这些特征称为属性。

（2）域（Domain）：每个属性的取值范围对应一个值的集合，称为该属性的域。

（3）目或度（Degree）：目或度指的是一个关系中属性的个数。

（4）候选码（Candidate Key）：若关系中的某一属性或属性组的值，能唯一地标识一个元组，则称该属性或属性组为候选码。

（5）主码（Primary Key）：或称主键，若一个关系有多个候选码，则选定其中一个作为主码。

（6）主属性（Prime Attribute）：包含在任何候选码中的属性称为主属性。

（7）外码（Foreign Key）：如果关系模式 R 中的属性或属性组不是该关系的码，但它是其他关系的码，那么该属性对关系模式 R 而言是外码。

（8）全码（All-key）：关系模型的所有属性组是这个关系模式的候选码，称为全码。

2. 关系代数运算

关系代数的运算符有 4 类：集合运算符、专门的关系运算符、算术比较符和逻辑运算符，系统架构设计师考试重点考查集合运算符与专门的关系运算符。集合运算符、专门的关系运算符的含义及解释详见表 8.3。

关系代数的运算中还有一类是属于扩展的关系代数运算，可以从基本的关系运算中导出。系统架构设计师考试关注的是外连接运算，这是连接运算的一种扩展。外连接运算包括左外连接、右外连接、完全外连接。具体见表 8.4。

表 8.3　集合运算符和专门的关系运算符的含义及解释

运算符		含义	解释
集合运算符	∪	并	关系 R 与 S 的并是由属于 R 或属于 S 的元组构成的集合
	−	差	关系 R 与 S 的差是由属于 R 但不属于 S 的元组构成的集合
	∩	交	关系 R 与 S 的交是由属于 R 同时又属于 S 的元组构成的集合
	×	笛卡儿积	两个关系分别为 *n* 列和 *m* 列的关系 R 和 S 的笛卡儿积是一个（*n*+*m*）列的元组的集合。其中的前 *n* 列是关系 R 的一个元组，后 *m* 列是关系 S 的一个元组，记作 R×S，如果 R 和 S 有相同的属性名，可在属性名前加关系名作为限定，以示区别。若 R 有 *K*1 个元组，S 有 *K*2 个元组，则 R 和 S 的笛卡儿积有 *K*1×*K*2 个元组
专门的关系运算符	σ	选择	取得关系 R 中符合条件的行
	π	投影	取得关系 R 中符合条件的列
	⋈	连接	等值连接：选取关系 R、S，取两者笛卡儿积中属性值相等的元组。 自然连接：一种特殊的等值连接，它要求比较的属性列必须是相同的属性组，并且把结果中重复属性去掉
	÷	除	给定关系 R(X，Y) 和 S(Y，Z)，其中 X，Y，Z 为属性组。R 中的 Y 与 S 中的 Y 可以有不同的属性名，但必须出自相同的域集。R 与 S 的除运算得到一个新的关系 P(X)，P 是 R 中满足下列条件的元组在 X 属性列上的投影：元组在 X 上分量值 *x* 的象集 Yx 包含 S 在 Y 上投影的集合

表 8.4　外连接运算符

运算符	含义	名词解释
⟕	左外连接	取出左侧关系中所有与右侧关系中任一元组都不匹配的元组，用空值 null 填充所有来自右侧关系的属性，构成新的元组，将其加入自然连接的结果中
⟖	右外连接	取出右侧关系中所有与左侧关系中任一元组都不匹配的元组，用空值 null 填充所有来自左侧关系的属性，构成新的元组，将其加入自然连接的结果中
⟗	完全外连接	完成左外连接和右外连接的操作。即填充左侧关系中所有与右侧关系中任一元组都不匹配的元组，并填充右侧关系中所有与左侧关系中任一元组都不匹配的元组，将产生的新元组加入自然连接的结果中

3. 关系数据库设计基本理论

（1）函数依赖：设 R(U)是属性 U 上的一个关系模式，X 和 Y 是 U 的子集，r 是 R 的任一关系，如果对于 r 中的任意两个元组 u 和 v，只要有 u[X]=v[X]，就有 u[Y]=v[Y]，则称 X 函数决定 Y，或称 Y 函数依赖于 X，记为 X→Y。函数依赖是一种最重要、最基本的数据依赖。而关系数据库设计理论的核心就是数据间的函数依赖。

（2）非平凡的函数依赖：如果 X→Y，Y⊈X，则称 X→Y 是非平凡的函数依赖。

（3）平凡的函数依赖：如果 X→Y，但 Y⊆X，则称 X→Y 是平凡的函数依赖。

（4）完全函数依赖：例如，有学生关系模式（学号，系号，系主任，课程号，成绩），该关系

模式的主码是学号+课程号，（学号，课程号）→成绩是完全函数依赖。

（5）部分函数依赖：上述例子中，（学号，课程号）→系号就属于部分函数依赖，因为对于系号来说有学号就可以推出系号。

（6）传递依赖：上述例子中，学号→系号，系号→系主任名，则称系主任名传递依赖于学号。

（7）函数依赖的公理系统（Armstrong 公理系统）：从已知的一些函数依赖，可以推导出另外一些函数依赖，这就需要一系列推理规则，这些规则常被称作“Armstrong 公理”。

设关系式 R(U，F)，U 是关系模式 R 的属性集，F 是 U 上的一组函数依赖，则有以下三条推理规则：

1）A1 自反律：若 Y⊆X⊆U，则 X→Y 为 F 所蕴含。

2）A2 增广律：若 X→Y 为 F 所蕴含，且 Z⊆U，则 XZ→YZ 为 F 所蕴含。

3）A3 传递律：若 X→Y，Y→Z 为 F 所蕴含，则 X→Z 为 F 所蕴含。

根据上面三条推理规则，又可推出下面三条推理规则：

1）合并规则：若 X→Y，X→Z，则 X→YZ 为 F 所蕴含。

2）伪传递规则：若 X→Y，WY→Z，则 XW→Z 为 F 所蕴含。

3）分解规则：若 X→Y，Z⊆Y，则 X→Z 为 F 所蕴含。

4. 关系数据库的规范化

关系数据库设计的方法之一就是设计满足适当范式的模式，通常可以通过判断分解后的模式达到几范式来评价模式的规范化程度。范式包括：1NF、2NF、3NF、BCNF、4NF、5NF。根据系统架构设计师考试的要求，这里重点介绍 1NF、2NF、3NF、BCNF 的基本概念。

（1）第一范式（1NF）：若关系模式 R 的每一个分量是不可再分的数据项，则关系模式 R 属于第一范式。

（2）第二范式（2NF）：若关系模式 R∈1NF，且每一个非主属性完全依赖主码时，则关系式 R 是 2NF（第二范式）。

（3）第三范式（3NF）：当 2NF 消除了非主属性对主码的传递函数依赖，则称为 3NF。

（4）BC 范式（BCNF）：如果关系模式 R∈1NF，且每个属性都不传递依赖于 R 的候选码，那么称 R 是 BCNF 模式。

上述 4 种范式之间有如下联系：BCNF⊂3NF⊂2NF⊂1NF。

5. 事务管理

DBMS 运行的基本工作单位是事务，事务是用户定义的一个数据库操作序列，这些操作序列要么全做，要么全都不做，是一个不可分割的工作单位。事务具有的四个特性（ACID）：

（1）原子性（Atomicity）：事务是数据库的逻辑工作单位，事务的所有操作在数据库中要么全做，要么全都不做。

（2）一致性（Consistency）：事务的执行使数据库从一个一致性状态变成另一个一致性状态。

（3）隔离性（Isolation）：一个事务的执行不能被其他事务干扰。

（4）持久性（Durability）：指一个事务一旦提交，它对数据库的改变必须是永久的，即便系

统出现故障时也是如此。

6. 相关的 SQL 语句

（1）BEGIN TRANSACTION：事务开始语句。

（2）COMMIT：事务提交语句，表示事务执行成功地结束，把事务对数据库的修改写入磁盘（事务对数据库的操作首先是在缓冲区中进行的）。

（3）ROLL BACK：事务回滚语句，表示事务执行不成功地结束，即把事务对数据库的修改进行恢复。

7. 并发控制

在多用户共享系统中，许多事务可能同时对同一数据进行操作，称为并发操作，此时数据库管理系统的并发控制子系统负责协调并发事务的执行，保证数据库的完整性不受破坏，同时避免用户得到不正确的数据。并发控制的主要技术是封锁，主要有两种类型的封锁，分别是 X 封锁和 S 封锁。

（1）排他型封锁（X 封锁）：如果事务 T 对数据 A（可以是数据项、记录、数据集以至整个数据库）实现了 X 封锁，那么只允许事务 T 读取和修改数据 A，其他事务要等事务 T 解除 X 封锁以后，才能对数据 A 实现任何类型的封锁。可见 X 封锁只允许一个事务独锁某个数据，具有排他性。

（2）共享型封锁（S 封锁）：如果事务 T 对数据 A 实现了 S 封锁，那么允许事务 T 读取数据 A，但不能修改数据 A，在所有 S 封锁解除之前决不允许任何事务对数据 A 实现 X 封锁。

8. 数据库的备份与恢复

（1）备份（转储）与恢复：备份是指通过数据转储和监理日志文件的方法监理冗余数据，DBA 定期地将整个数据库复制到磁带或另一个磁盘上保存起来的过程。这些备用的数据文本称为后备副本。数据库的恢复是指把数据库从错误状态恢复到某一个已知的正确状态的功能。当数据库遭到破坏后，就可以利用后备副本把数据库恢复，这时数据库只能恢复到备份时的状态，从那以后的所有更新事务必须重新运行才能恢复到故障时的状态。

（2）备份分类包括以下 4 种：

1）静态备份：指备份期间不允许（或不存在）对数据库进行任何存取、修改活动。静态备份简单，但备份必须等待用户事务结束才能进行，新的事务必须等待备份结束才能执行。这降低了数据库的可用性。

2）动态备份：指备份期间允许对数据库进行存取或修改，即备份和用户事务可以并发执行。动态备份可克服静态备份的缺点，但备份结束时后援副本上的数据并不能保证正确有效。

3）海量备份：指每次备份全部数据库。

4）增量备份：指每次只备份上次备份后更新过的数据。如果数据库很大，事务处理又十分频繁，则增量备份方式是很有效的。

（3）数据库的 4 类故障：事务故障、系统故障、介质故障、计算机病毒。事务故障的恢复有两个操作：撤销事务（UNDO）和重做事务（REDO）。介质故障的恢复由数据库管理员装入数据库的副本和日记文件副本，再由系统执行撤销和重做操作。

8.3 数据库设计

【基础知识点】

（1）数据库设计的基本步骤。可以分为用户需求分析、概念结构设计、逻辑结构设计、物理结构设计、应用程序设计、运行维护。

（2）数据需求分析。用户需求分析是综合各用户的应用需求，对现实世界要处理的对象进行详细调查，在了解先行系统的概况，确定新系统功能的过程中，协助用户明确对新系统的信息要求、处理要求和系统要求，确定新系统的边界。

（3）概念结构设计。概念数据模型又称为实体-联系模型，它按照用户的观点来对数据和信息建模，主要用于数据库设计。概念模型主要用实体-联系方法（Entity-Relationship Approach）表示，简称 E-R 方法。概念结构设计工作步骤包括：选择局部应用、逐一设计分 E-R 图和 E-R 图合并。在进行 E-R 图合并时，需解决属性冲突、命名冲突和结构冲突。E-R 模型简称 E-R 图，是描述概念世界、建立概念模型的实用工具。E-R 图的三个要素有：

1）实体（型）：用矩形框表示，框内标注实体名称。

2）属性：用椭圆形表示，并用连线与实体连接起来。

3）实体之间的联系：用菱形框表示，框内标注联系名称，用连线将菱形框分别与有关实体相连，并在连线上注明联系类型。

（4）逻辑结构设计。逻辑结构设计阶段主要工作步骤包括确定数据模型、将 E-R 图转换成指定的数据模型、确定完整性约束和确定用户视图。

（5）物理设计。物理设计主要工作步骤包括确定数据分布、存储结构和访问方式。

（6）数据库实施。数据库实施是根据逻辑和物理设计的结果，在计算机上建立实际的数据库结构，数据加载（装入），进行试运行和评价。

（7）数据库运行维护。数据库运行维护主要包括对数据库性能的监测和改善、故障恢复、数据库的重组和重构。在数据库运行阶段，对数据库的维护主要由 DBA 完成。

（8）商业智能。商业智能（Business Intelligence，BI）是企业对商业数据的搜集、管理和分析的系统过程，目的是使企业的各级决策者获得知识或洞察力，帮助他们作出对企业更有利的决策。一般认为数据仓库、联机分析处理（OLAP）和数据挖掘是商业智能的三大组成部分。

（9）数据仓库（Data Warehouse）是一个面向主题的、集成的、相对稳定且随时间变化的数据集合，用于支持管理决策。数据仓库的关键特征是：面向主题、集成的、非易失的、时变的。

传统数据库与数据仓库的比较见表 8.5。

OLTP 与 OLAP 的比较：OLTP 即联机事务处理，就是我们经常说的关系数据库的基础；OLAP 即联机分析处理，是数据仓库的核心部分。OLTP 与 OLAP 的具体区别见表 8.6。

表 8.5　传统数据库与数据仓库的比较

比较项目	传统数据库	数据仓库
数据内容	当前值	历史的、归档的、归纳的、计算的数据（处理过的）
数据目标	面向业务操作程序、重复操作	面向主体域，分析应用
数据特性	动态变化、更新	静态、不能直接更新，只能定时添加、更新
数据结构	高度结构化、复杂，适合操作计算	简单、适合分析
使用频率	高	低
数据访问量	每个事务一般只访问少量记录	每个事务一般访问大量记录
对响应时间的要求	计时单位小，如秒	计时单位相对较大，除了秒，还有分钟、小时

表 8.6　OLTP 与 OLAP 的区别

项目	OLTP	OLAP
用户	操作人员、低层管理人员	决策人员、高级管理人员
功能	日常操作处理	分析决策
DB 设计	面向应用	面向主题
数据	当前的、最新的、细节的、二维的、分立的	历史的、聚集的、多维的、集成的、统一的
存取	读/写数十条记录	读上百万条记录
工作单位	简单的事务	复杂的查询
用户数	上千个	上百个
DB 大小	100MB 至 GB 级	100GB 至 TB 级

（10）数据挖掘。数据挖掘是在没有明确假设的前提下去挖掘信息、发现知识。数据挖掘所得到的信息应具有先知、有效和实用三个特征。先前未知的信息是指该信息是预先未曾预料到的，即数据挖掘是要发现那些不能靠直觉发现的信息或知识，甚至是违背直觉的信息或知识，挖掘出的信息越是出乎意料，就可能越有价值。

8.4　应用程序与数据库的交互

【基础知识点】

常见应用程序与数据库交互方式有库函数、嵌入式 SQL、通用数据接口标准和对象关系映射等。

1. 库函数级别访问接口

库函数级别的数据访问接口是数据库提供的最底层的高级程序语言访问数据接口，如 Oracle 数据库的 Oracle Call Interface（OCI）。OCI 由一组应用程序开发接口（API）组成，实际是将结构化查询语言（SQL）和高级语言程序相结合。其缺点是强依赖于特定的数据库，需数据库开发人员

对数据库机制有较深的理解，学习难度较大，开发效率不是很高。

2. 嵌入式 SQL 访问接口

嵌入式 SQL（Embeded SQL）是一种将 SQL 语句直接写入某些高级程序语言源代码中的方法。

3. 通用数据接口标准

开放数据库连接（Open DataBase Connectivity，ODBC）是为解决异构数据库间的数据共享产生的。优点是不依赖于任何 DBMS，能以统一的方式处理所有的关系数据库。常见数据库接口包括数据库访问对象（Data Access Object，DAO）、远程数据库对象（Remote Data Object，RDO）、ActiveX 数据对象（Active Data Object，ADO）、Java 数据库连接（java DataBase Connection，JDBC）。

4. ORM 访问接口

对象关系映射（Object Relationship Mapping，ORM）用于实现面向对象编程语言里不同类型系统数据之间的转换。典型的 ORM 框架有 Hibernate、Mybatis 和 JPA 等。

（1）Hibernate：全自动化的框架，强大、复杂、笨重、学习成本高。

（2）Mybatis：半自动的框架。

（3）JPA：Java 自带的框架。

8.5 NoSQL 数据库

【基础知识点】

1. 分类与特点

NoSQL 是一种概念，泛指非关系型的数据库，区别于关系数据库，它们不保证关系数据的 ACID 特性。NoSQL 数据库按照所使用的数据结构的类型，可分为列式存储数据库、键值对存储数据库、文档型数据库、图数据库。

（1）列式存储数据库用来应对分布式存储的海量数据，键仍然存在，特点是指向了多个列。现有产品如 Cattandra、HBate、Riak。

（2）键值对存储数据库优势是简单、易部署。但是只对部分值查询或更新时效率低下。现有产品如 Tokzo Cabinet/Tzranu、Redis、Voldemort、Oracle BDB。

（3）文档型数据库在处理网页等复杂数据时，比传统键值对数据库查询效率更高，现有产品如 CouchDB、MongoDB、SequoiaDB。

（4）图数据库适合存储通过图进行建模的数据，如社交网络数据、生物信息网络数据，交通网络数据等。常见产品有 Neo4J、InfoGrid、Infinite Graph 等。

NoSQL 特征：易扩展、大数据量、高性能、灵活的数据模型、高可用。

2. 体系框架

NoSQL 整体框架由下至上分为数据持久层（Data Persistence）、数据分布层（Data Distribution Model）、数据逻辑模型层（Data Logical Model）和接口层（Interface）。

NoSQL 数据库适用情况：数据模型比较简单、需灵活性更强的 IT 系统、对数据库性能要求较高、不需要高度的数据一致性。

8.6 分布式数据库

【基础知识点】

1. 体系结构

分布式数据库的体系结构如图 8.3 所示。

（1）全局视图：全局视图（全局外模式）是全局应用的用户视图，是全局概念模式的子集，该层直接与用户（或应用程序）交互。

（2）全局概念模式：全局概念模式定义分布式数据库中数据的整体逻辑结构，数据就如同根本没有分布一样，可用传统的集中式数据库中所采用的方法进行定义。

（3）分片模式：将一个关系模式分解成为几个数据片。

（4）分配模式：分布式数据库的本质特性就是数据分布在不同的物理位置。分配模式的主要职责是定义数据片段（即分片模式的处理结果）的存放节点。

（5）局部概念模式：局部概念模式是局部数据库的概念模式。

（6）局部内模式：局部内模式是局部数据库的内模式。

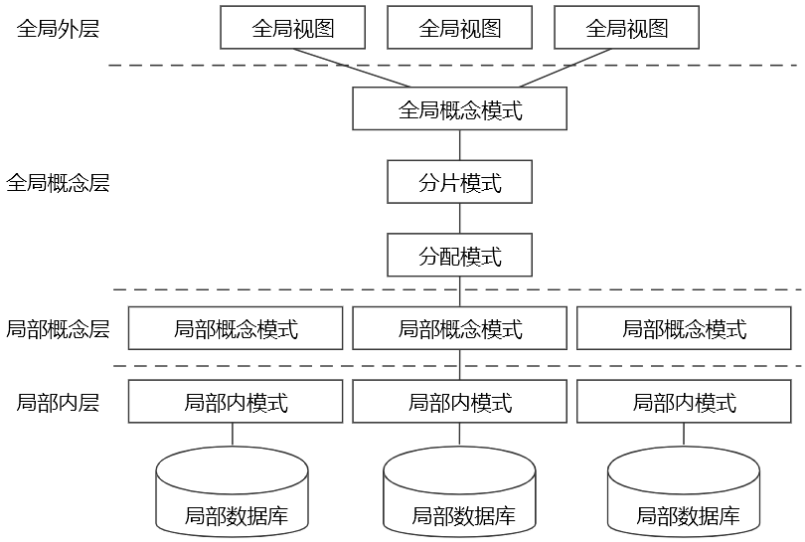

图 8.3　分布式数据库体系结构

2. 特点

分布式数据库具有如下特点：

（1）共享性：不同的节点的数据共享。

（2）自治性：每个节点对本地数据都能独立管理。

（3）可用性：某一场地故障时，可以使用其他场地上的副本而不至于使整个系统瘫痪。

（4）分布性：数据分布在不同场地上存储。

3. 分布透明性

分布透明性是指用户不必关心数据的逻辑分片，不必关心数据存储的物理位置分配细节，也不必关心局部场地上数据库的数据模型。分布透明性包括分片透明性、位置透明性和局部数据模型透明性。

（1）分片透明性是分布透明性的最高层次。所谓分片透明性是指用户或应用程序只对全局关系进行操作而不必考虑数据的分片。

（2）位置透明性是分布透明性的下一层次。所谓位置透明性是指用户或应用程序应当了解分片情况，但不必了解片段的存储场地。

（3）局部数据模型透明性（逻辑透明）是指用户或应用程序应当了解分片及各片断存储的场地，但不必了解局部场地上使用的是何种数据模型。

8.7 数据库优化技术

【基础知识点】

1. 集中式数据库优化技术

集中式数据库性能优化最常见的是反规范化设计，主要包括增加冗余列、增加派生列、重新组表、水平分割表、垂直分割表。

（1）增加冗余列。增加冗余列是指在多个表中具有相同的列，它常用来在查询时避免连接操作。

（2）增加派生列。增加派生列指增加的列可以通过表中其他数据计算生成。它的作用是在查询时减少计算量，从而加快查询速度。

（3）重新组表。重新组表指如果许多用户需要查看两个表连接出来的结果数据，则把这两个表重新组成一个表来减少连接，从而提高性能。

（4）水平分割表。按记录进行分割，把数据放到多个独立的表中，主要用于表数据规模很大、表中数据相对独立或数据需要存放到多个介质上时使用。

（5）垂直分割表。对表进行分割，将主键与部分列放到一个表中，主键与其他列放到另一个表中，在查询时减少 I/O 次数。

反规范化设计的优点是避免进行表之间的连接操作，从而可以提高数据操作的性能；缺点是会造成数据的重复存储，浪费了磁盘空间，会产生数据的不一致性问题。若要避免数据不一致的问题，可以通过设置触发器、采用事务机制（适用于单体数据库中）、应用保证（适用于异构数据库之间）以及批处理脚本的方式，这些方式的优缺点这里不再赘述。

2. 分布式数据库优化技术

分布式数据库的性能优化可以采用主从复制、读写分离、分表、分库等技术。

（1）主从复制。主从复制是建立一个和主数据库完全一样的数据库环境，称为从数据库。这样做的好处是：

1）做数据的热备。作为后备数据库，主数据库服务器故障后，可切换到从数据库继续工作，避免数据丢失。

2）架构的扩展。业务量越来越大，I/O 访问频率过高，单机无法满足，此时做多库的存储，降低磁盘 I/O 访问的频率，提高单个机器的 I/O 性能。

3）读写分离。使数据库能支持更大的并发。

在 MySQL 数据库中，主从数据库同步的模式有全同步、半同步、异步三种方式。主从数据库之间通过 binlog（二进制日志）进行数据的同步。具体过程如图 8.4 所示。

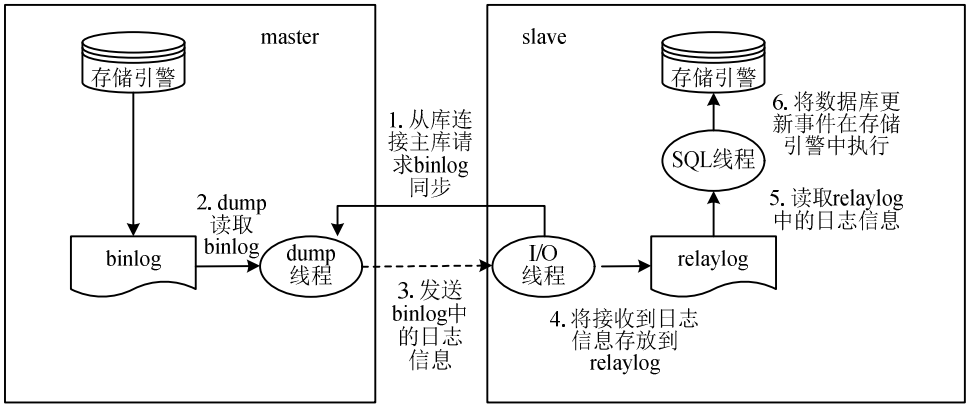

图 8.4　MySQL 主从复制过程

这里的 binlog 日志有 3 种模式：

1）基于 SQL 语句的复制：每一条更新的语句（insert、update、delete）都会记录在 binlog 中，进而同步到从库的 relaylog 中，被从库的 SQL 线程取出来，回放执行。该模式的优点是 binlog 的日志量可能会比较少，比如一个涉及行数为 1000 行的 update 语句，同步这一个语句，就同步了 1000 行的数据。缺点是同步的 SQL 语句里如果含有绑定本地变量的函数、关键字，可能造成主从不一致的情况。比如 SQL 语句中有 time 函数，如果主从数据库的服务器时间不是精确相等，就会造成结果不一致。

2）基于行的复制：不记录 SQL 语句，只记录哪个记录更新前和更新后的数据，可以保证主从之间的数据绝对相同。缺点是：1 条 SQL 更新 1000 行的数据无法再偷懒，必须原原本本同步 1000 行的数据量。

3）混合复制：以上两种模式的混合，选取两者的优点。对于有绑定本地特性、评估可能造成主从不一致的 SQL 语句，则自动选用基于行的复制，其他的选择基于 SQL 语句的复制。

（2）读写分离。设置不同的主、从数据库分别负责不同的操作。让主数据库负责数据的写操作，从数据库负责数据的读操作。通过角色分担的策略，分别提升读写性能，有效减少数据并发操作的延迟。

（3）分表。分表也叫分片，可以提升数据库并发以及 I/O 的性能。分表重在单个实例内部，将一张大表分成若干小表，业务同时访问多个表。分表的方式有两种，见表 8.7。

表 8.7　分表的方式

垂直切分	水平切分
把一个大表切分为多个表。例如交易 ID、状态、用户、金额、商品等，作为一个热表；另外的交易备注、物流信息等众多其他属性可作为另一个表	把一个大表分为多个表，每个表都包含相同列，例如交易 ID、状态、用户、金额、商品等，但都包含不同的行。例如一个表包含的是交易 ID 从 1 到 999999 的交易数据，另一个表包含的是交易 ID 从 1000000 到 9999999 的交易数据

（4）分库。分库是将原本存放在一个实例上众多分类的数据（表），分开存放到不同的实例上。有利于差异化管理。例如，一个简单的电商网站，包括用户、商品、订单三个业务模块，可以将用户数据、商品数据、订单数据分开放到三台不同的数据库服务器上，而不是将所有数据都放在一台数据库服务器上。

8.8　分布式缓存技术 Redis

【基础知识点】

1. 基本概念

Redis 是一种分布式缓存技术，在本书 8.5 节中也是一种键值对数据库类型。Redis 用作缓存组件时，其基于内存的读写特性，比基于磁盘读写的数据库性能要高很多，适合缓存高频热点的数据，来提高读性能。这样可以降低对数据库服务器的查询请求，提高系统性能。Redis 以 key-value（键值对）的形式为数据的保存格式。键（key）可以是一个字符串，值（value）可以是任意类型的数据。如整型、字符型、数组、列表、集合等。例如键值对：（“20231234”,“张珂”），其 key：“20231234”是该数据的唯一标识，而 value：“张珂”是该数据实际存储的内容。

2. 数据类型

Redis 支持的数据类型主要包括 string 类型、hash 类型、set 类型、list 类型、zset 类型、pub/sub 类型。

（1）string 类型：是 Redis 基本类型。可用于缓存层或计数器，如视频播放量、文章浏览量等。

（2）hash 类型：代替 string 类型，节省空间。描述用户信息较为方便。

（3）set 类型：无序集合，每个值不能重复。可用于去重、抽奖、初始化用户池等。

（4）list 类型：双向链表结构，可以模拟栈、队列等形式。可用于回复评论、点赞。

（5）zset 类型：有序集合、每个元素有一个分数。如首页推荐 10 个最热门的帖子。

3. 访问方式

引入 Redis 后，热点数据存放在 Redis 中，但由于存在“一份数据存放了多个位置”，所以要考虑数据的一致性问题。读写数据的基本步骤为：

（1）读数据：①根据 key 读缓存；②读取成功则直接返回；③若 key 不在缓存中，则根据 key 读数据库；④读取成功后，写缓存；⑤成功返回。

（2）写数据：①根据 key 值写数据库；②成功后更新缓存 key 值；③成功返回。

4. 过期策略

在使用 Redis 时，一般会设置 Redis 缓存空间的大小，不会让数据无限制地存放到 Redis 中，对于设置了过期时间的数据可以采用两种方式去淘汰这些数据。

（1）定期删除。Redis 每隔一段时间就会抽取一些设置了过期时间的 key。这里的抽取是随机进行的，因为无法对所有的 key 进行遍历，会给系统带来很大的负担。但是这样也会导致一些 key 到了过期时间也仍然没有被删除。

（2）惰性删除。查询 key 的时候 Redis 会对 key 进行检测，发现如果已经达到过期时间，则删除。惰性删除的缺点是如果这些过期的 key 没有被访问，那么它们就一直无法被删除，而且一直占用内存。

除了上述两种方式，Redis 又提供了一些淘汰机制，主要有：

- volatile-lru（最近最少使用）：从已设置过期时间的 key 中，移出最近最少使用的 key 进行淘汰。
- volatile-lfu（最不经常使用）：从 key 中选择最不经常使用的进行淘汰。
- volatile-random（随机淘汰算法）：从已设置过期时间的 key 中随机选择 key 淘汰。
- volatile-ttl（生存时间淘汰）：从已设置过期时间的 key 中，移出将要过期的 key。
- allkeys-lru：从所有 key 中选择最近最少使用的进行淘汰。
- allkeys-lfu：从所有 key 中选择最不经常使用的进行淘汰。
- allkeys-random：从所有 key 中随机选择 key 进行淘汰。

5. 数据持久化

在实际应用中，一旦服务器宕机，内存中的数据将全部丢失。我们很容易想到的一个解决方案是，从后端数据库恢复这些数据，但这种方式存在两个问题：一是，需要频繁访问数据库，会给数据库带来巨大的压力；二是，这些数据是从慢速数据库中读取出来的，性能肯定比不上从 Redis 中读取来得快，这会导致使用这些数据的应用程序响应变慢。所以，对 Redis 来说，实现数据的持久化，避免从后端数据库中进行恢复，是至关重要的。Redis 数据持久化的方式有两种，见表 8.8。

6. 缓存异常问题

Redis 在提高数据查询效率与保护数据库方面都起到了至关重要的作用，但是在实际应用中可能会出现 Redis 异常的情况，这里总结了一些常见的异常问题与对应的解决方案。

表 8.8　Redis 持久化方式

项目	RDB 内存快照（Redis Data Base）	AOF 日志（Append Only File）
说明	把当前内存中的数据集快照写入磁盘（数据库中所有键值对数据）。恢复时是将快照文件直接读到内存里	通过持续不断地保存 Redis 服务器所执行的更新命令来记录数据库状态，类似 MySQL 的 binlog 日志。恢复数据时需要从头开始回放更新命令
磁盘刷新频率	低	高
文件大小	小	大
数据恢复效率	高	低
数据安全	低	高

（1）缓存穿透。大量请求访问了没有缓存的 key，即大量的 key 在 Redis 里是不存在的，从而导致请求直接访问数据库，数据库压力增大。可能的原因如下：

1）恶意攻击，造成大量访问不存在的 key。例如登录时使用无效的用户名，在软考网站查询成绩时输入不存在的身份证号、准考证号。

解决方案：①针对比较少的请求来源 IP，主动限制其访问次数，或者拉入黑名单；②应用程序来检查 key 的合法性，提前拒绝不合法的请求；③使用布隆过滤器。

2）大量请求访问数据库里有但 Redis 没有的 key。例如新业务刚刚上线，此时 Redis 是空的。

解决方案：①预热 Redis，运行一个批处理脚本，将可能会大量访问的数据预先加载到 Redis，业务再“开张”；②在最前端进行流量控制，逐步把请求释放进来。给出一段时间，让 Redis 逐步加载热数据；③如果是在数据库里也没有的 key，也需要在 Redis 中设置 key，使其值为 null 或空。

（2）缓存雪崩。大量请求访问到缓存中的 key，这些 key 是存在的，但同时到了过期时间，从而导致请求直接访问数据库，数据库压力增大。缓存雪崩可能进而影响一系列的雪崩，影响到上下游的所有应用服务。可能的原因如下：

1）Redis 故障。比如 Redis 宕机，网络出现抖动等。

解决方案：①使用主从复制提高可用性，使用 cluster 集群方案降低故障时影响的范围；②如果出现故障，则可以采取服务降级、熔断、限流等措施。

2）大量的 key 采用了相同的过期时间。例如在同一时刻设置了大量的 key，但过期时间都是 5 分钟。

解决方案：过期时间加上一个随机值，使得众多 key 均匀过期。

（3）缓存击穿。少量热点的 key 缓存时间失效了，使得请求直接访问数据库。

可能的原因：热点的 key 设置了太短的过期时间。例如秒杀业务下的“库存数量”。

解决方案：①将 key 设置较长的过期时间。对于非常重要的 key，则设置永久有效。但需要解决好与数据库中的 key 的一致性问题；②使用分布式锁。如果热点 key 失效了，要控制好访问后端数据库的流量。只允许一个请求去访问数据库，取出最新的 key，存放到 Redis，其他请求则必须

等待。但分布式锁也要防止出现异常的情况。

7. Redis 集群

Redis 也可以采用集群的方式部署，主要有主从复制集群、哨兵集群、Cluster 集群方式。集群切片的方式主要分为客户端分片、代理分片、服务器端分片三种方式。

8.9 练习题

1．数据库系统与文件系统的区别不包括（　　）。

A．对应用程序的高度独立性　　B．数据的充分共享性

C．文件组织形式的多样化　　D．操作方便性

解析：数据库对数据的存储是按照同一种数据结构进行的，不同的应用程序都可以直接操作这些数据（即对应用程序的高度独立性）。数据库系统对数据的完整性、一致性和安全性都提供了一套有效的管理手段（数据的充分共享性）。数据库系统还提供管理和控制数据的各种简单操作命令，容易掌握，使用户编写程序简单（即操作方便性）。

答案：C

2．（　　）描述的是 DBMS 向用户提供数据操纵语言，实现对数据库中数据的基本操作，如检索、插入、修改和删除。

A．数据定义　　B．数据库操作

C．数据库运行管理　　D．数据组织、存储与管理

解析：DBMS 功能主要包括数据定义、数据库操作、数据库运行管理、数据组织、存储与管理、数据库的建立和维护。其中数据库操作是 DBMS 向用户提供数据操纵语言，实现对数据库中数据的基本操作，如检索、插入、修改和删除。

答案：B

3．给定关系模式 R(U,F)，其中：属性集 U={A1,A2,A3,A4,A5,A6}；函数依赖集 F={A1→A2，A1→A3，A3→A4，A1A5→A6}。关系模式 R 的候选码为__（1）__，由于 R 存在非主属性对码的部分函数依赖，所以 R 属于__（2）__。

（1）A．A1A3　　B．A1A4　　C．A1 A5　　D．A1A6

（2）A．1NF　　B．2NF　　C．3NF　　D．BCNF

解析：判断候选码有一种比较快速的方式，就是看哪个属性只在依赖集 F 中“→”的左边出现过，那么该关系的候选码就必定包含那个属性。很显然选项 C 中 A1 和 A5 都是满足要求的，所以题干给定关系模式的候选码就是 A1A5。对于空（2），“R 存在非主属性对码的部分函数依赖”说明不满足 2NF 的要求，那么该关系模式只能是 1NF。

答案：C　A

4．某互联网文化发展公司因业务发展，需要建立网上社区平台，为用户提供一个对网络文化产品（如互联网小说、电影、漫画等）进行评论、交流的平台。该平台的部分功能如下：

（a）用户帖子的评论计数器；

（b）支持粉丝列表功能；

（c）支持标签管理；

（d）支持共同好友功能等；

（e）提供排名功能，如当天最热前 10 名帖子排名、热搜榜前 5 排名等；

（f）用户信息的结构化存储；

（g）提供好友信息的发布/订阅功能。

该系统在性能上需要考虑高性能、高并发，以支持大量用户的同时访问。开发团队经过综合考虑，在数据管理上决定采用 Redis＋数据库（缓存＋数据库）的解决方案。

【问题】Redis 支持丰富的数据类型，并能够提供一些常见功能需求的解决方案。请选择题干描述的（a）～（g）功能选项，填入表 8.9 中（1）～（5）的空白处。

表 8.9　Redis 数据类型与业务功能对照表

数据类型	存储的值	可实现的业务功能
string	字符串、整数或浮点数	（1）
list	列表	（2）
set	无序集合	（3）
hash	包括键值对的无序散列表	（4）
zset	有序集合	（5）

解析：Redis 支持的数据类型主要包括 string 类型、hash 类型、set 类型、list 类型、zset 类型。

（1）string 类型：是 Redis 基本类型。可用于缓存层或计数器，如视频播放量、文章浏览量等。

（2）hash 类型：代替 string 类型，节省空间。描述用户信息较为方便。

（3）set 类型：无序集合，每个值不能重复。可用于去重、抽奖、初始化用户池等。

（4）list 类型：双向链表结构，可以模拟栈、队列等形式。可用于回复评论、点赞。

（5）zset 类型：有序集合、每个元素有一个分数。如首页推荐 10 个最热门的帖子。

答案：

（1）（a）　（2）（b）（g）　（3）（c）（d）　（4）（f）　（5）（e）

5．引入 Redis 后，热点数据存放在 Redis 中，但由于存在“一份数据存放了多个位置”，所以要考虑数据的一致性问题。读写数据的基本步骤是什么？

答案：读写数据的基本步骤为：

（1）读数据：①根据 key 读缓存；②读取成功则直接返回；③若 key 不在缓存中，则根据 key 读数据库；④读取成功后，写缓存；⑤成功返回。

（2）写数据：①根据 key 值写数据库；②成功后更新缓存 key 值；③成功返回。